

BAM 3034 Lab 1 Python Refresher

Student: Olamide Olorunfemi

PDF Export of Completed Jupyter Notebook

This PDF contains the complete code and saved output for both required parts of the lab. Part A reads height and weight data from hw.txt, calculates BMI, and prints the BMI groups. Part B demonstrates the eight required Python text-data topics.

The matching editable notebook file is BAM3034_Lab1_Olamide_Olorunfemi.ipynb. The values shown below were executed using the supplied hw.txt file.

Part A BMI Analysis Code

""""BAM 3034 - Lab 1: Python Refresher
Student: Olamide Olorunfemi

Part A reads height/weight records from hw.txt, calculates BMI, then groups people by BMI category. Part B demonstrates the required text-data functions.
""""

```
from pathlib import Path
import re
```

```
# -----
# Part A - BMI analysis from hw.txt
# -----
def load_height_weight_records(filename="hw.txt"):
    """Return a list of (height_cm, weight_lb) records from the input file."""
    values = [float(line.strip()) for line in Path(filename).read_text().splitlines()
               if line.strip()]
    if len(values) % 2 != 0:
        raise ValueError("hw.txt must have one height and one weight for each person.")
    return list(zip(values[0::2], values[1::2]))

def calculate_bmi(height_cm, weight_lb):
    """Calculate BMI after converting centimetres to metres and pounds to kg."""
    height_m = height_cm / 100
    weight_kg = weight_lb / 2.2
    return weight_kg / (height_m ** 2)

def bmi_category(bmi):
    """Return the BMI category required in the lab instructions."""
    if bmi < 18.5:
        return "Under Weight"
    if bmi < 25:
        return "Normal"
    if bmi < 30:
        return "Over Weight"
    return "Obesity"

def analyse_bmi_file(filename="hw.txt"):
    records = load_height_weight_records(filename)
    groups = {"Under Weight": [], "Normal": [], "Over Weight": [], "Obesity": []}

    for person_number, (height_cm, weight_lb) in enumerate(records, start=1):
        bmi = calculate_bmi(height_cm, weight_lb)
        groups[bmi_category(bmi)].append((person_number, bmi))

    for category, people in groups.items():
        print(f"{category}:")
        for person_number, bmi in people:
            print(f"Person- {person_number} : {bmi:.4f}")
        print(f"Group Size: {len(people)}")
        if people:
            group_average = sum(bmi for _, bmi in people) / len(people)
            print(f"Average BMI for this group: {group_average:.4f}")
        print()

    all_bmis = [bmi for people in groups.values() for _, bmi in people]
    print(f"The file has records for: {len(records)} persons")
    print(f"Average BMI for all these {len(records)} persons: {sum(all_bmis) / len(all_bmis):.4f}")
    return groups
```

Part A BMI Analysis Output

Under Weight:

Person- 5 : 18.2379

Person- 7 : 14.6092

Person- 9 : 17.3737

Group Size: 3

Average BMI for this group: 16.7403

Normal:

Person- 12 : 24.4261

Person- 13 : 23.5923

Group Size: 2

Average BMI for this group: 24.0092

Over Weight:

Person- 1 : 25.2525

Person- 3 : 28.3108

Person- 11 : 28.3795

Group Size: 3

Average BMI for this group: 27.3143

Obesity:

Person- 2 : 31.4564

Person- 4 : 44.0631

Person- 6 : 44.4444

Person- 8 : 50.0877

Person- 10 : 69.5733

Group Size: 5

Average BMI for this group: 47.9250

The file has records for: 13 persons

Average BMI for all these 13 persons: 32.2929

Part B Text Data Code

```
# -----
# Part B - Working with text data using Python
# -----
def text_data_examples():
    print("1. STRING TYPES")
    text = "Sentiment analysis with Python"
    print(type(text))
    print('He said, "Python is useful."')
    print("First line\nSecond line")
    print()

    print("2. STRING OPERATIONS")
    first_word = "text"
    second_word = "mining"
    print(first_word + " " + second_word)
    print("ha" * 3)
    print("Python" in text)
    print()

    print("3. STRING INDEXING AND SLICING")
    word = "analysis"
    print(word[0])      # first character
    print(word[-1])     # last character
    print(word[0:4])    # characters 0 to 3
    print(word[:2])     # every second character
    print()

    print("4. STRING IMMUTABILITY")
    original = "python"
    # original[0] = "P" would cause a TypeError because strings cannot change in place.
    updated = "P" + original[1:]
    print(original)
    print(updated)
    print()

    print("5. USEFUL STRING METHODS")
    message = " Python makes TEXT analysis easier. "
    print(message.strip())
    print(message.lower())
    print(message.upper())
    print(message.replace("easier", "clearer"))
    print(message.count("a"))
    print(message.find("TEXT"))
    print(message.startswith(" Py"))
    print()

    print("6. STRING SPLITTING AND JOINING")
    sentence = "Text mining finds useful patterns"
    words = sentence.split()
    print(words)
    print("-".join(words))
    csv_values = "positive,negative,neutral"
    print(csv_values.split(","))
    print()

    print("7. STRING FORMATTING")
    student = "Olamide"
    score = 92.4567
    print("{} scored {:.2f}% on the lab.".format(student, score))
    print(f"{student} scored {score:.2f}% on the lab.")
    print("{} completed {}.".format(name=student, part="Part B"))
    print()

    print("8. REGULAR EXPRESSIONS")
    review = "Email support@example.com or call 613-555-0199 on 2026-09-20."
    print(re.findall(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}", review))
    print(re.findall(r"\d{3}-\d{3}-\d{4}", review))
    print(re.findall(r"\d{4}-\d{2}-\d{2}", review))
    print(re.sub(r"\d", "#", review))
```

Part B Text Data Output

1. STRING TYPES

```
<class 'str'>
He said, "Python is useful."
First line
Second line
```

2. STRING OPERATIONS

```
text mining
hahaha
True
```

3. STRING INDEXING AND SLICING

```
a
s
anal
aayi
```

4. STRING IMMUTABILITY

```
python
Python
```

5. USEFUL STRING METHODS

```
Python makes TEXT analysis easier.
python makes text analysis easier.
PYTHON MAKES TEXT ANALYSIS EASIER.
Python makes TEXT analysis clearer.
4
15
True
```

6. STRING SPLITTING AND JOINING

```
['Text', 'mining', 'finds', 'useful', 'patterns']
Text-mining-finds-useful-patterns
['positive', 'negative', 'neutral']
```

7. STRING FORMATTING

```
Olamide scored 92.46% on the lab.
Olamide scored 92.46% on the lab.
Olamide completed Part B.
```

8. REGULAR EXPRESSIONS

```
['support@example.com']
['613-555-0199']
['2026-09-20']
Email support@example.com or call ###-###-#### on ####-##-##.
```